Übung 3

Einstieg in Java

Programmierung 1 · WS 2026/27
Bachelor Cyber Security & Privacy · André Kless, M.Sc.

Ziel: Die Java-Umgebung einrichten und die bekannten Grundbausteine in lauffähige Java-Programme übertragen.

Arbeitsweise: Nutzen Sie die Folien und Syntaxdiagramme aus Kapitel 3. Arbeiten Sie mit Java 25 oder höher und `void main()`. Ab Aufgabe 2 dürfen Sie eine IDE verwenden.

Reihenfolge: Basic (1–4) festigt die Grundlagen, Combined (5–8) verbindet sie, Challenge (9–10) fordert eigene Programmentwürfe.

Eingaben: Bei Zahleneingaben dürfen Sie korrekt geschriebene ganze Zahlen im Bereich von `int` voraussetzen. Prüfen Sie zusätzlich die in der Aufgabe geforderten Wertebereiche.

01 Java einrichten und im Terminal starten | Basic

a) Installieren Sie das Oracle JDK 25 LTS oder eine höhere Version mithilfe der Installationsfolie. Prüfen Sie im Terminal mit `java --version` und `javac --version`, ob beide Befehle verfügbar sind und mindestens Version 25 anzeigen.

b) Legen Sie im Terminal einen Arbeitsordner an und wechseln Sie hinein. Erstellen Sie mit einem Texteditor die Datei `Hallo.java`. Schreiben Sie eine kompakte `void main()` mit einer Ausgabe durch `IO.println()`: `Hallo Java!`

c) Übersetzen Sie die Datei mit `javac Hallo.java` und starten Sie sie mit `java Hallo`. Welche Datei entsteht beim Übersetzen? Was enthält sie, und welche Aufgabe hat die JVM beim Start?

d) Ändern und speichern Sie den Ausgabetext. Starten Sie zunächst nur mit `java Hallo`, danach erneut mit vorherigem Übersetzen. Erklären Sie den Unterschied der Ausgaben.

02 In der IDE weiterarbeiten | Basic

a) Richten Sie IntelliJ IDEA oder eine andere Java-IDE ein. Legen Sie ein Projekt `Prog1` mit JDK 25 oder höher an. Erstellen Sie in `src` ein Java Compact File `Aufgabe02.java` ohne Package. Verwenden Sie später je Aufgabe eine eigene Datei, z. B. `Aufgabe03.java`.

b) Schreiben und starten Sie ein Programm, das Ihren Namen, Ihren Studiengang und einen selbst gewählten Gruß in drei Zeilen ausgibt. Verwenden Sie `void main()` und `IO.println()`. Rücken Sie den Rumpf mit vier Leerzeichen ein und ergänzen Sie einen erklärenden `//`-Kommentar.

c) Entfernen Sie testweise das Semikolon einer Ausgabe. Lesen Sie die Fehlermeldung und korrigieren Sie den Fehler. Ändern Sie danach nur den ausgegebenen Text: Warum ist das allein kein Syntaxfehler?

Basic: Variablen und Selektion

03 Einen Einkauf berechnen | Basic

Ein Heft kostet 2,50 Euro. Das Programm soll einen Namen und eine Stückzahl von 1 bis 100 einlesen und den Gesamtpreis ausgeben. Für diese Aufgabe dürfen Sie eine Stückzahl im gültigen Bereich voraussetzen.

a) Lesen Sie den Namen mit `IO.readln()` als `String` ein. Lesen Sie die Stückzahl ebenfalls mit `IO.readln()` ein und wandeln Sie sie mit `Integer.parseInt()` in `int` um. Geben Sie jeweils eine verständliche Eingabeaufforderung aus.

b) Deklarieren Sie den Stückpreis als `final double STUECKPREIS = 2.5;`. Berechnen Sie den Gesamtpreis in einer Variablen. Speichern Sie außerdem in einer `boolean`-Variablen, ob mindestens zehn Hefte bestellt wurden. Geben Sie Name, Stückzahl, Gesamtpreis und diesen Wahrheitswert aus. Eine Ausgabe wie `7.5 Euro` genügt.

c) Deklarieren Sie den Gesamtpreis zunächst ohne Initialisierung und weisen Sie ihm den berechneten Wert danach zu. Zeigen Sie zusätzlich eine gleichwertige Deklaration mit Initialisierung. Ordnen Sie beide Schreibweisen den Wegen im Syntaxdiagramm zur Deklaration zu.

d) Testen Sie mit den Stückzahlen 1, 3, 9, 10 und 100. Notieren Sie vorher die erwarteten Preise und Wahrheitswerte und vergleichen Sie diese mit der Programmausgabe.

04 Schaltjahre bestimmen | Basic

Für diese Aufgabe gilt ein vereinfachtes Kalendermodell: Vor 1582 verwenden wir die julianische Regel, ab 1582 die gregorianische. Historisch erfolgte der Kalenderwechsel nicht überall gleichzeitig. Sie dürfen eine eingegebene Jahreszahl von 1 bis 9999 voraussetzen.

Julianisch ist jedes durch 4 teilbare Jahr ein Schaltjahr. Gregorianisch muss das Jahr durch 4 teilbar sein; durch 100 teilbare Jahre sind jedoch keine Schaltjahre, außer sie sind auch durch 400 teilbar.

a) Lesen Sie die Jahreszahl ein. Bestimmen Sie zunächst mit verschachtelten `if`-/`else`-Anweisungen, ob ein Schaltjahr vorliegt. Speichern Sie das Ergebnis in `boolean schaltjahr` und geben Sie es aus. Verwenden Sie `%` zur Prüfung der Teilbarkeit.

b) Ersetzen Sie die gesamte Fallunterscheidung zur Schaltjahrberechnung durch einen einzigen logischen Ausdruck mit `&&` und `||`. Testen Sie beide Varianten mit 1500, 1582, 1600, 1700, 1900, 2000, 2024 und 2025. Notieren Sie die erwarteten Ergebnisse vor dem Test.

Basic: Schaltjahre und ein Fehler aus der Praxis

04 Schaltjahre bestimmen — Fortsetzung | Basic

c) Untersuchen Sie den folgenden Fehlerfall im Debugger. Bearbeiten Sie diesen Teil nach den Schleifen und dem Debugging in Aufgabe 6 und 7.

Am 31. Dezember 2008 blieben Zune-30-Musikplayer wegen eines Fehlers in der Datumsumrechnung hängen. Der folgende Java-Code bildet den veröffentlichten fehlerhaften Schleifenablauf nach. Für den direkten Test beginnt er bei 2008 statt wie das Original bei 1980; die Schaltjahrprüfung steht direkt als Ausdruck im Code.

`tage` zählt ab dem 1. Januar des Startjahres, wobei dieser Tag die Nummer 1 hat. Das Programm soll ganze Jahre abziehen und anschließend das zugehörige Jahr sowie die Tagesnummer innerhalb dieses Jahres ausgeben. Ab 2008 gilt die gregorianische Regel.

```
void main() {
    int jahr = 2008;
    int tage = 366;
    while (tage > 365) {
        boolean schaltjahr = jahr % 4 == 0
                && (jahr % 100 != 0 || jahr % 400 == 0);
        if (schaltjahr) {
            if (tage > 366) {
                tage -= 366;
                jahr++;
            }
        } else {
            tage -= 365;
            jahr++;
        }
    }
    IO.println("Jahr: " + jahr + ", Tag im Jahr: " + tage);
}
```

Starten Sie in Ihrer IDE mit einem Breakpoint bei der Berechnung von `schaltjahr`. Spielen Sie die Startwerte 365, 366 und 367 für `tage` jeweils schrittweise durch; `jahr` startet immer bei 2008. Prüfen Sie auch, ob der Breakpoint erreicht wird. Beobachten Sie die Variablen und Bedingungen. Falls sich die Werte wiederholen, beenden Sie den Lauf über „Stop“. Erklären und korrigieren Sie den Fehler. Prüfen Sie danach die drei Startwerte erneut. Warum genügt es nicht, nur `tage > 366` in `tage >= 366` zu ändern?

Zeitgenössischer Pressebericht: The Guardian — Bericht vom 1. Januar 2009

Combined: Mehrfachselektion und Iteration

05 Ein Menü mit zwei Schreibweisen | Combined

Ein Informationsmenü bietet drei Ausgaben an: 1 steht für „Öffnungszeiten“, 2 für „Kontakt“ und 3 für „Anfahrt“. Jede andere Zahl soll die Meldung „Ungültige Auswahl“ erzeugen. Pro Programmstart wird genau eine Auswahl eingelesen.

a) Schreiben Sie das Programm mit einer Mehrfachselektion durch `switch`, der Doppelpunktform und `break`. Geben Sie bei jeder gültigen Auswahl nur den zugehörigen Text aus.

b) Entfernen Sie testweise das `break` nach dem ersten Fall. Sagen Sie die Ausgabe für Auswahl 1 voraus und prüfen Sie sie. Erklären Sie, was bei dieser Schreibweise nach einem passenden Fall geschieht. Stellen Sie danach die korrekte Variante wieder her.

c) Schreiben Sie eine zweite Version mit der Pfeilform. Speichern Sie die Varianten in `Aufgabe05Doppelpunkt.java` und `Aufgabe05Pfeil.java`. Testen Sie beide mit 1, 2, 3 und 9. Begründen Sie, welche Schreibweise Sie für dieses Menü bevorzugen.

06 Ein Zahlenratespiel entwickeln | Combined

Der Computer wählt eine geheime ganze Zahl von 1 bis einschließlich `n`. Sie haben höchstens zehn Versuche, sie zu erraten. Schreiben Sie das Programm in `Aufgabe06.java`.

Beispielverlauf: Hier wurde zufällig die 7 gewählt. Fett gedruckte Zahlen sind Eingaben.

```
Obere Grenze n: 10
Dein Tipp: 5
Deine Zahl ist zu klein.
Dein Tipp: 9
Deine Zahl ist zu groß.
Dein Tipp: 7
Richtig! Benötigte Versuche: 3
```

a) Lesen Sie `n` ein. Wiederholen Sie die Eingabe mit einem Hinweis, solange `n < 1` ist. Erzeugen Sie danach einmalig die geheime Zahl mit der vorgegebenen Zeile:

```
int geheimzahl = (int) (Math.random() * n) + 1;
```

`Math.random()` liefert eine Zufallszahl vom Typ `double` ab 0.0 und kleiner als 1.0. Nach der Multiplikation mit `n` entfernt `(int)` den Nachkommateil. `+ 1` verschiebt den Bereich auf 1 bis einschließlich `n`. Übernehmen Sie diese Zeile; sie gehört vor die Rateschleife.

b) Lesen Sie in einer Schleife die Tipps ein. Jeder eingelesene Tipp zählt als Versuch. Geben Sie bei einem falschen Tipp „Deine Zahl ist zu klein.“ oder „Deine Zahl ist zu groß.“ aus. Bei einem Treffer endet das Spiel sofort mit einer Erfolgsmeldung und der Anzahl benötigter Versuche. Nach zehn falschen Tipps endet es mit der Ausgabe der gesuchten Zahl.

c) Ersetzen Sie zum Testen die Zufallszahl vorübergehend durch einen festen Wert. Prüfen Sie: Treffer beim ersten Versuch, Treffer beim zehnten Versuch und zehn falsche Tipps. Testen Sie beide Richtungen der Rückmeldung sowie ungültige Werte für `n`. Stellen Sie danach die Zufallszahl-Erzeugung wieder her und testen Sie den Sonderfall `n = 1`.

Combined: Programme untersuchen und erweitern

07 Einen Schleifenfehler im Debugger finden | Combined

Das folgende Programm soll die Zahlen von 1 bis einschließlich 5 addieren. Es lässt sich übersetzen, liefert aber nicht das gewünschte Ergebnis.

```
void main() {
    int summe = 0;
    for (int i = 1; i < 5; i++) {
        summe += i;
    }
    IO.println(summe);
}
```

a) Bestimmen Sie ohne Ausführung das gewünschte und das tatsächliche Ergebnis. Ist dies ein Syntaxfehler oder ein Fehler in der Programmlogik? Begründen Sie Ihre Einordnung.

b) Öffnen Sie das Programm in Ihrer IDE. Setzen Sie in IntelliJ durch einen Klick in den linken Rand neben `summe += i;` einen Breakpoint und starten Sie mit „Debug“. Beobachten Sie bei jedem Halt `i` und `summe`; führen Sie die Addition schrittweise aus. Notieren Sie die Werte vor und nach jeder Addition. In einer anderen IDE nutzen Sie die entsprechende Debugger-Funktion.

c) Korrigieren Sie die Schleifenbedingung. Prüfen Sie im Debugger, dass nun auch 5 addiert wird. Entfernen Sie den Breakpoint und testen Sie das vollständige Programm.

d) Erweitern Sie das korrigierte Programm: Bei `i == 3` soll `continue` die Addition überspringen. Sagen Sie das neue Ergebnis voraus und prüfen Sie es. Ersetzen Sie anschließend nur `continue` durch `break` und erklären Sie die geänderte Wirkung.

08 Punkte eines Gitters ausgeben | Combined

Ein Gitter hat vier Spalten und drei Zeilen. Die Koordinaten beginnen bei 0: `x` läuft von 0 bis 3, `y` von 0 bis 2. Die Punkte sollen zeilenweise ausgegeben werden, zunächst alle Punkte mit `y = 0`.

a) Schreiben Sie zwei geschachtelte `for`-Schleifen: außen für `y`, innen für `x`. Geben Sie jeden Punkt in einer eigenen Zeile im Format `(x,y)` aus, ohne Leerzeichen nach dem Komma. Notieren Sie vor dem Start die vollständige erwartete Ausgabe.

b) Ergänzen Sie einen Zähler und geben Sie nach beiden Schleifen die Anzahl der ausgegebenen Punkte aus. Wo muss dieser Zähler deklariert werden, damit er an allen benötigten Stellen sichtbar ist?

c) Überspringen Sie nun alle Punkte mit `x == y` durch `continue` in der inneren Schleife. Der Zähler soll nur tatsächlich ausgegebene Punkte zählen. Welche Punkte entfallen, und welche Anzahl muss am Ende erscheinen?

Challenge: Bekannte Algorithmen und eigene Suche

09 Den ggT für drei Zahlen berechnen | Challenge

Der ggT-Algorithmus aus der Vorlesung verarbeitet zwei positive ganze Zahlen durch wiederholtes Subtrahieren. Erweitern Sie das Vorgehen auf drei Zahlen `a`, `b` und `c`. Verwenden Sie dabei den Zusammenhang `ggT(a,b,c) = ggT(ggT(a,b),c)`.

a) Lesen Sie jede der drei Zahlen ein. Akzeptieren Sie nur Werte von 1 bis 1000; wiederholen Sie bei einem ungültigen Wert die Eingabe dieser Zahl. Bewahren Sie die drei ursprünglichen Werte für die abschließende Ausgabe auf.

b) Berechnen Sie zunächst den ggT von `a` und `b`, danach den ggT dieses Ergebnisses und `c`. Arbeiten Sie mit Hilfsvariablen und dem Subtraktionsverfahren aus der Vorlesung. Schreiben Sie beide Berechnungsphasen direkt in `main`; eigene Methoden sind hier noch nicht erforderlich.

c) Geben Sie die drei Ausgangszahlen und ihren ggT aus. Testen Sie mit (18,12,9), (7,7,7), (1,9,6) und (21,14,35). Notieren Sie jeweils das erwartete Ergebnis sowie das Zwischenergebnis nach der ersten Berechnungsphase.

d) Zählen Sie zusätzlich die Subtraktionen in beiden Phasen getrennt. Vergleichen Sie (18,12,9) mit (12,18,9). Erklären Sie außerdem am Zweizahlenfall (0,5), warum die Prüfung auf positive Zahlen für dieses Verfahren wichtig ist.

10 Passende Packungen finden | Challenge

Ein Versanddienst bietet Packungen für genau 4 oder genau 6 Artikel an. Für eine Bestellung mit `anzahl` Artikeln sollen alle Möglichkeiten gefunden werden, die Artikel ohne freie Plätze und ohne Rest auf diese Packungen zu verteilen. Jede Packungsart darf auch nullmal vorkommen; die Reihenfolge der Packungen spielt keine Rolle.

a) Lesen Sie eine ganze Artikelanzahl von 1 bis 100 ein und wiederholen Sie ungültige Eingaben. Planen Sie eine Suche mit zwei geschachtelten Schleifen. Begründen Sie sinnvolle Obergrenzen für die Anzahl der Vierer- und Sechserpackungen.

b) Prüfen Sie jede Kombination genau einmal. Geben Sie nur passende Kombinationen mit beiden Packungsanzahlen aus. Zählen Sie die gefundenen Möglichkeiten. Wenn keine passt, soll stattdessen eine eindeutige Meldung erscheinen. Testen Sie mit 1, 4, 6, 8, 12 und 24.

c) Erstellen Sie eine zweite Variante, die beim ersten Treffer beide Suchschleifen beendet. Verwenden Sie dafür eine Sprungmarke und `break`. Lassen Sie die Anzahl der Sechserpackungen außen und die der Viererpackungen innen jeweils aufsteigend ab 0 laufen. Welcher Treffer wird für 24 zuerst gefunden?

d) Erläutern Sie, weshalb ein `break` ohne Sprungmarke in der inneren Schleife dafür nicht genügt. Prüfen Sie auch in der zweiten Variante, dass bei einer unmöglichen Artikelanzahl die Meldung aus b) erscheint.